



Smart Expense Tracker — Process Step by Step

This document explains **exactly what happens** at every step a user takes in the app, and **exactly how to deploy** the full app (web + mobile) from zero to live.

PART 1 — USER FLOW: Step by Step



STEP 0 — User Opens the App

User opens browser → types `your-app.vercel.app`

OR

User opens installed PWA from Android home screen

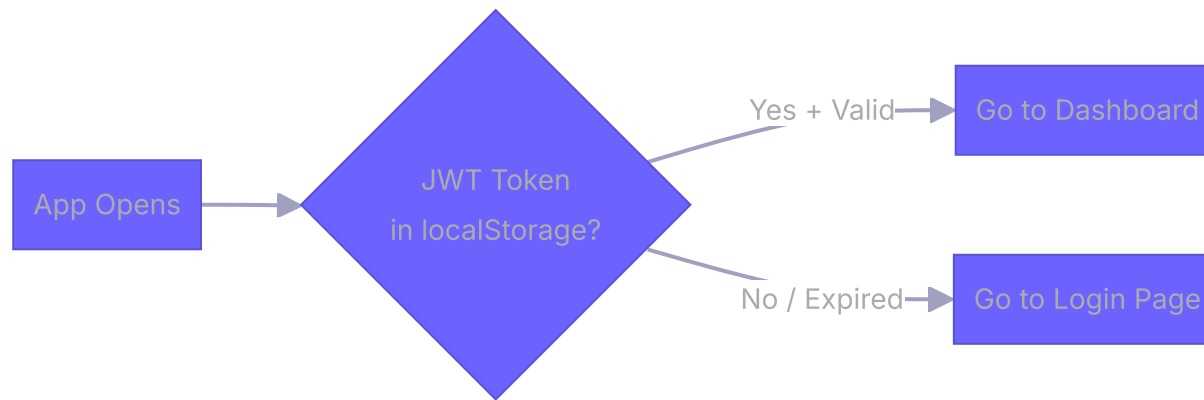
OR

User opens React Native app from Play Store

What happens under the hood:

- Browser/phone loads the React frontend from **Vercel's CDN**
- React Router checks if the user has a valid **JWT token** in `localStorage`

- If token exists and is not expired → goes directly to Dashboard
- If no token → redirect to Login page



● STEP 1 — Registration / Login

1A. New User — Register with Email

User fills: Name | Email | Password → clicks "Sign Up"

What happens:

1. React sends `POST /api/auth/register` to FastAPI backend
2. FastAPI validates the data (Pydantic schema)
3. Password is **hashed with bcrypt** — never stored as plain text

4. New row created in `users` table in **PostgreSQL (Supabase)**
5. A **JWT access token** (15 min) + **refresh token** (7 days) are created
6. Tokens sent back to React → stored in localStorage
7. User is redirected to Dashboard

1B. Existing User — Login

User enters Email + Password → clicks "Login"

What happens:

1. `POST /api/auth/login` called
2. FastAPI finds user by email in DB
3. **bcrypt.verify** compares entered password with stored hash
4. If match → new JWT token generated and returned
5. React stores token → redirect to Dashboard

1C. Google OAuth Login (One-click)

User clicks "Continue with Google"

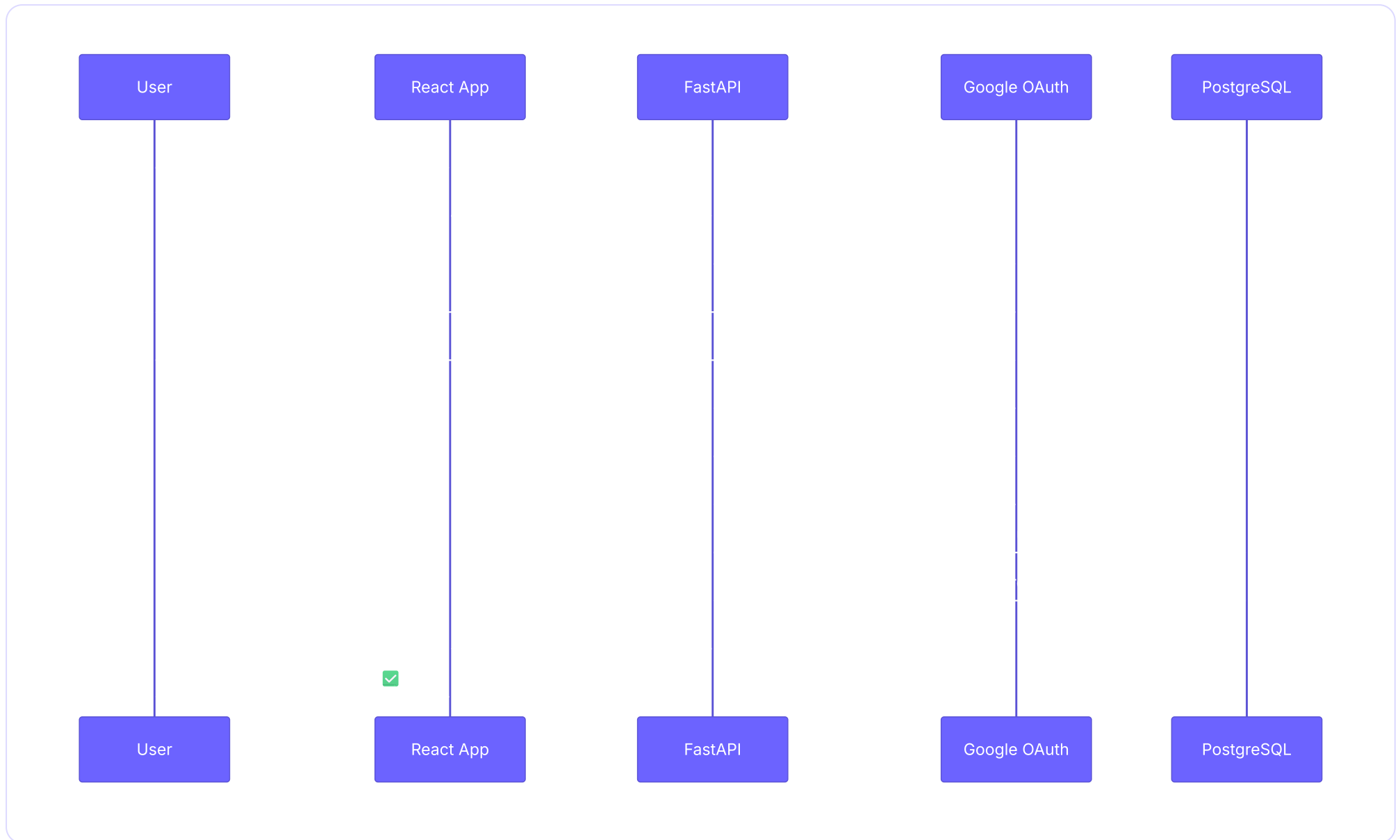
What happens:

1. React redirects to `GET /api/auth/google`
2. FastAPI redirects to **Google's OAuth consent screen**
3. User approves → Google sends back a `code` to your backend
4. FastAPI exchanges `code` for user's **Google profile** (name, email, photo)

5. If user already exists → log in | If new → create account

6. JWT token generated → returned to frontend

7. Redirected to Dashboard ✓



● STEP 2 — Dashboard (Home Screen)

After login, user lands on the **Dashboard**. This is loaded by calling:

```
GET /api/dashboard/summary      → total spent this month, income, savings
GET /api/dashboard/breakdown    → % split by category (Food 40%, Travel 20%...)
GET /api/dashboard/trend        → day-by-day spending for this month
GET /api/insights               → AI-generated insights (loaded separately)
GET /api/budgets                → budget limits and % consumed
```

What the user sees:

- 🏠 Total spent this month card
- 📊 Pie chart: category breakdown
- 📈 Bar chart: daily spending trend
- 🏠 Financial Health Score (0–100)
- 💡 AI Insight cards ("You spent 25% more on food this week")
- ⚠️ Budget warning banners if any category > 80%

*All chart data comes from the backend which **queries PostgreSQL**, aggregates with SQL, and returns JSON.*

● STEP 3 — Adding an Expense (4 Ways)

WAY 1 — Manual Entry (Simplest)

User clicks "+" button → fills form:

Amount: 350

Category: Food

Date: Today

Note: Dinner at Barbeque Nation

→ clicks "Save"

What happens:

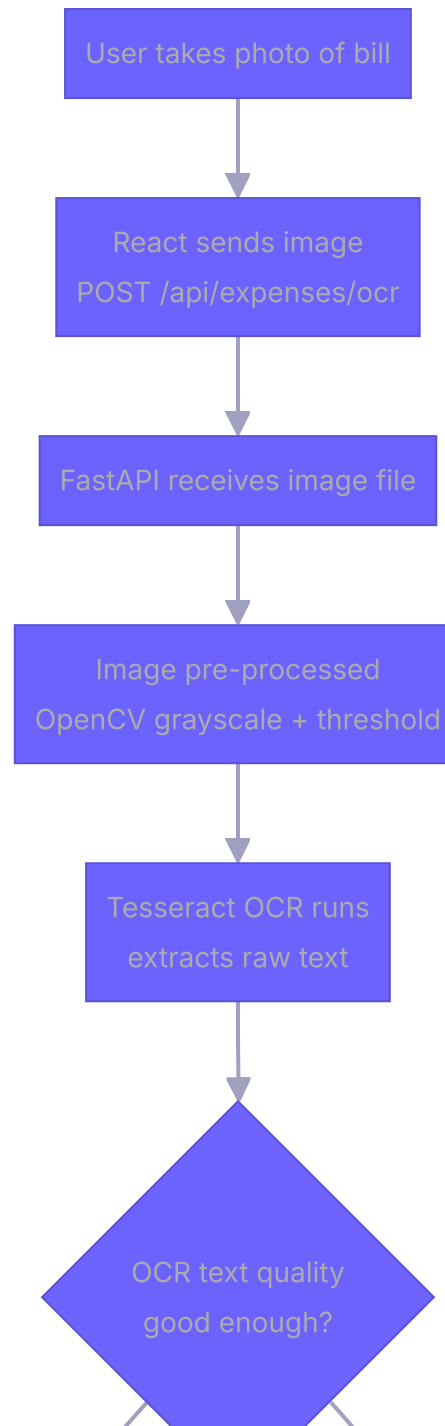
1. React sends `POST /api/expenses` with the form data
 2. FastAPI validates with Pydantic (ensures amount > 0, date is valid, etc.)
 3. If category is left blank → **AI auto-categorization kicks in** (see Step 4)
 4. New row inserted into `expenses` table in PostgreSQL
 5. Response sent back → Dashboard **re-fetches** and updates charts
-

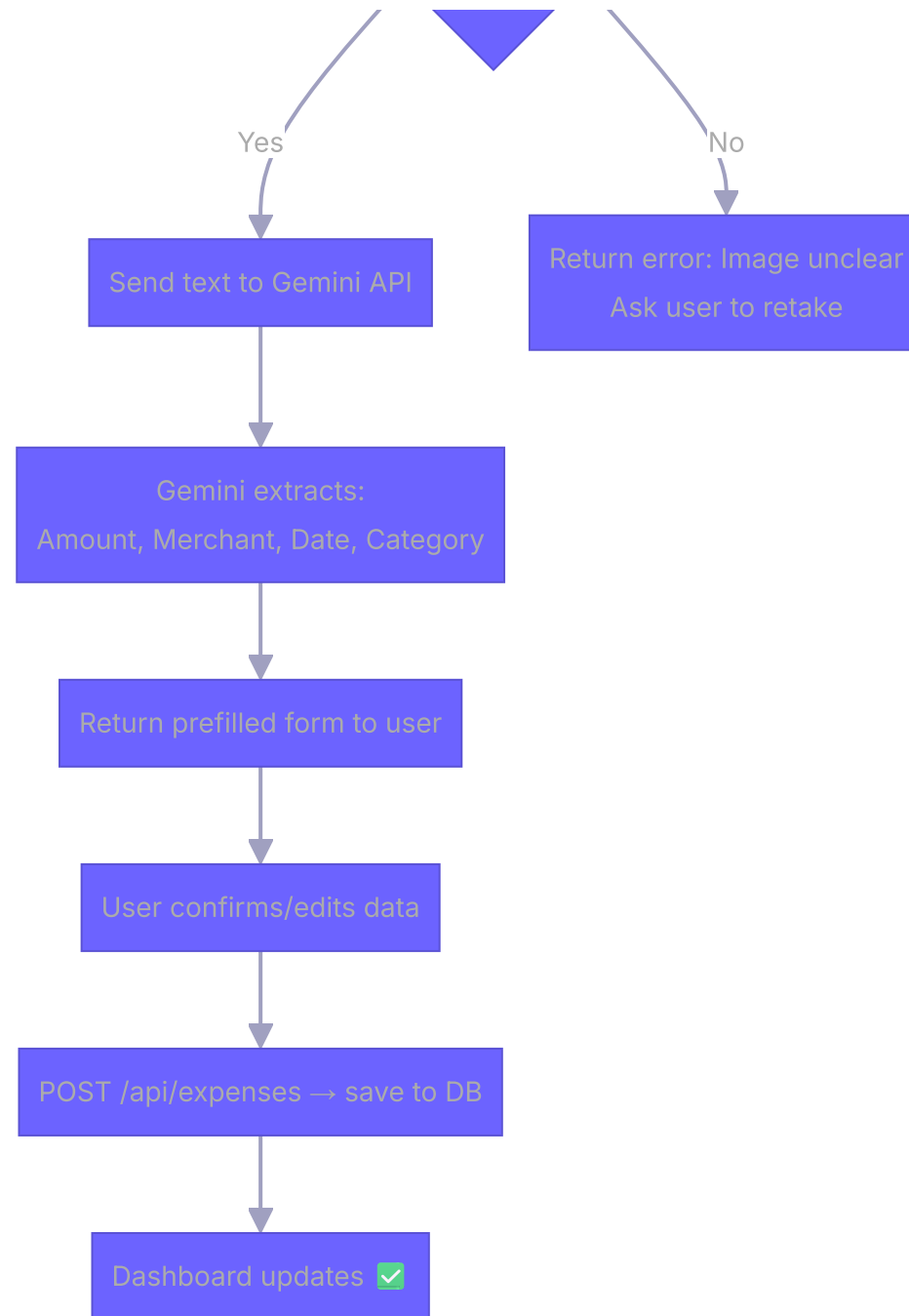
WAY 2 — OCR Receipt Scanner (Photo of Bill)

User clicks "Scan Receipt" → takes photo of bill

→ uploads image

What happens step by step:





What Gemini does with OCR text:

OCR raw text: "SWIGGY ORDER TOTAL Rs. 485 DATE 04-04-2026 ORDER#12345"

Gemini prompt: "Extract: amount, merchant, date, category from this text"

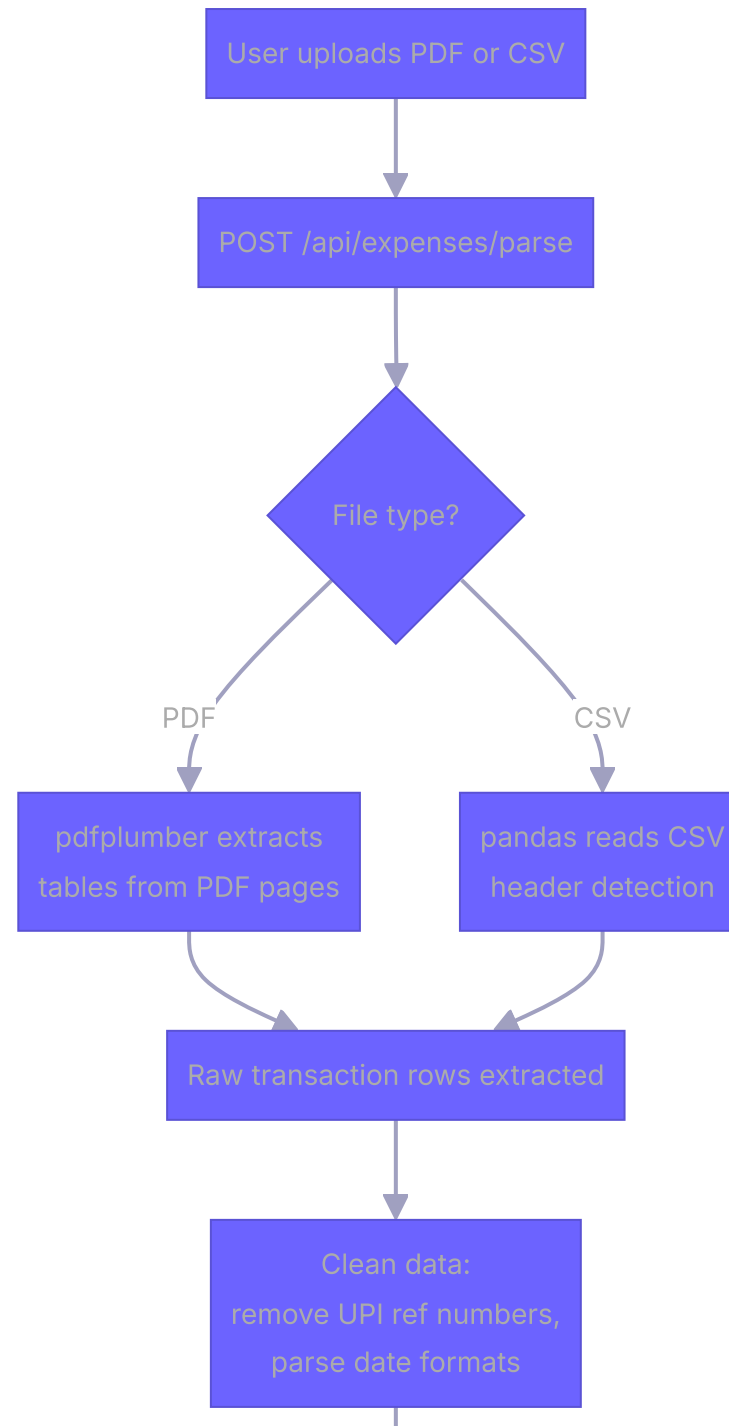
```
Gemini output: {  
  "amount": 485,  
  "merchant": "Swiggy",  
  "date": "2026-04-04",  
  "category": "Food & Dining"  
}
```

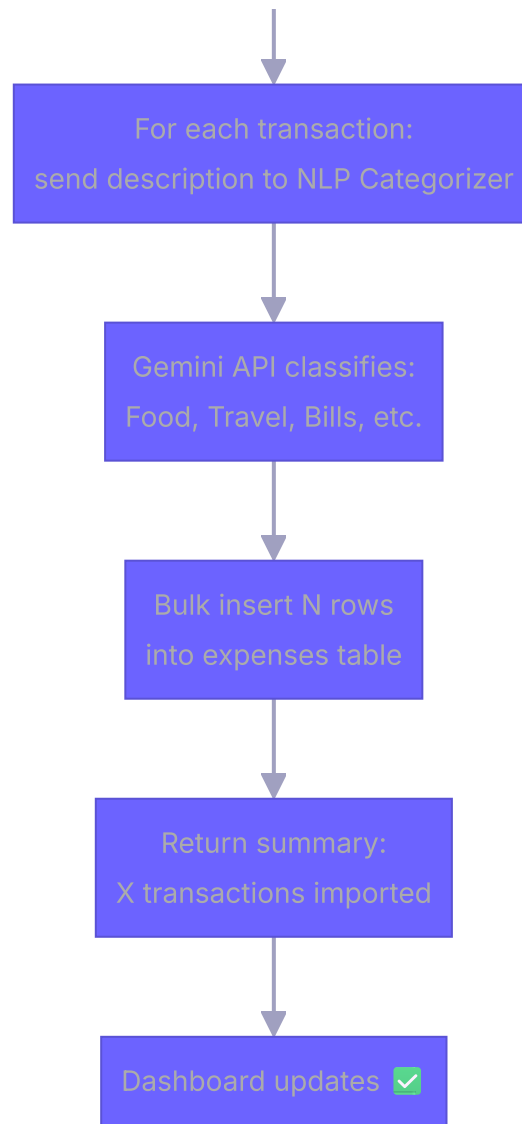
User sees a prefilled form → confirms → saved 

WAY 3 — Bank Statement Upload (PDF or CSV)

User goes to "Import" section →
uploads SBI/HDFC statement PDF or CSV

What happens step by step:





Example:

```
PDF row: "05/04 UPI-ZOMATO-9823xx DR 320.00"
```

↓

```
NLP output: { category: "Food & Dining", amount: 320, date: "2026-04-05" }
```

↓

```
Saved to DB 
```

⚡ This can import **100s of transactions in seconds** — that's the demo wow moment.

WAY 4 — AI Chat Entry (Natural Language)

```
User types: "spent 500 on Uber last night"  
            or "bought groceries for 1200 yesterday"
```

What happens:

1. `POST /api/chat` sends message to FastAPI
2. FastAPI sends to **Gemini API** with a system prompt:

```
You are an expense parser. Extract: amount, category, date, description.  
User said: "spent 500 on Uber last night"  
Today is 2026-04-05.  
Return JSON only.
```

1. Gemini returns:

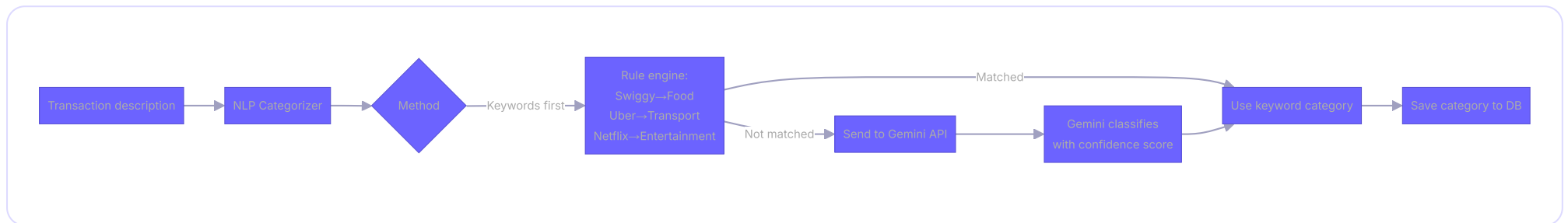
JSON

```
{ "amount": 500, "category": "Transport", "date": "2026-04-04", "description": "Uber ride" }
```

1. FastAPI auto-creates the expense entry
2. Chat replies: "✅ Logged ₹500 for Transport on Apr 4"

● STEP 4 — AI Auto-Categorization (Runs in background)

Every time an expense is added **without a category**, or imported from a statement:

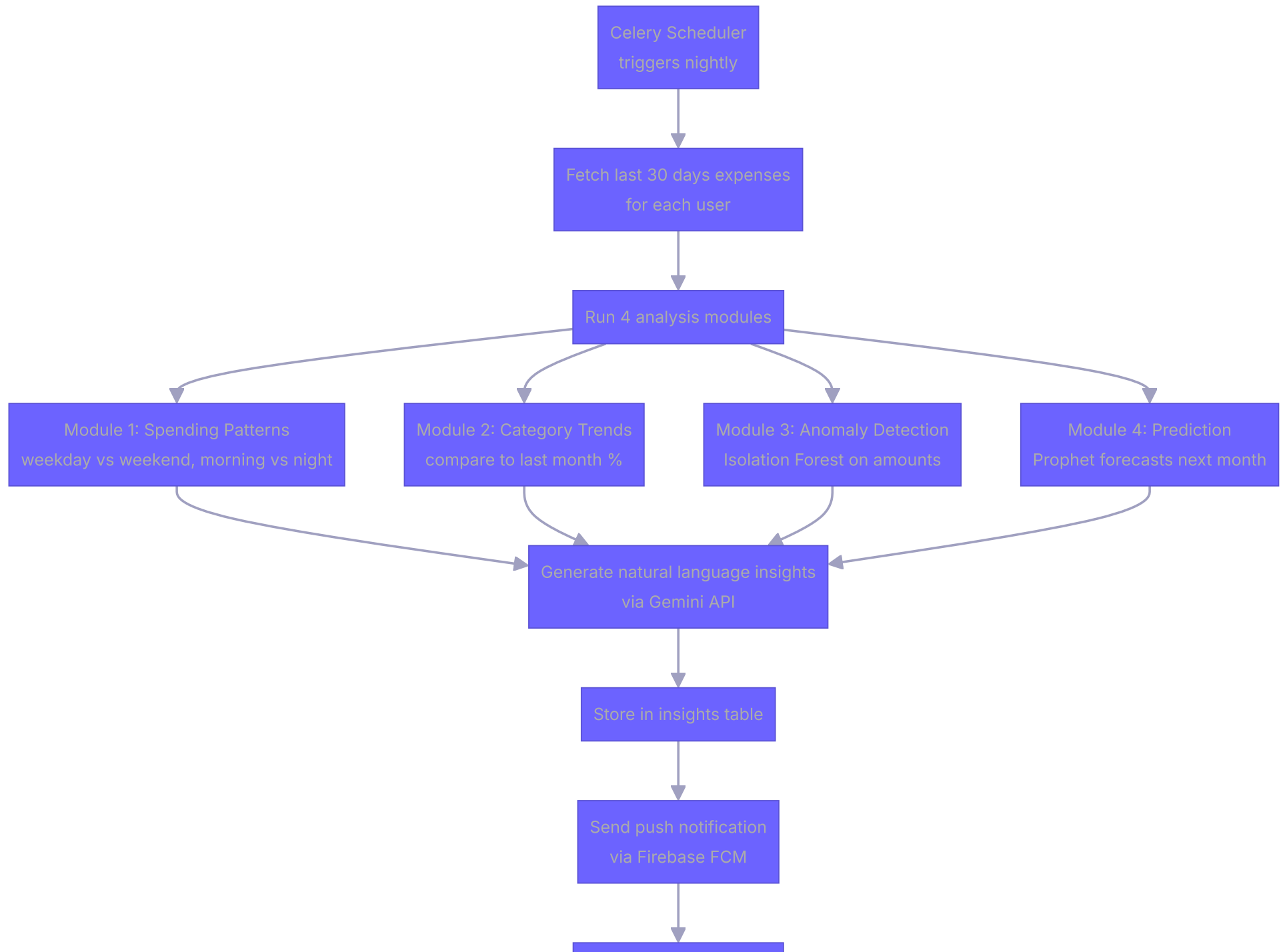


Categories recognized:

Food & Dining | Transport | Shopping | Entertainment | Bills & Utilities | Healthcare | Education | Travel | Investments | Other

● STEP 5 — AI Insights Engine

Every night at **midnight** (Celery scheduled task):



User sees insight card
on next Dashboard visit

Example Insights generated:

- 💡 "Your Food spending is ₹3,200 this month — 28% higher than last month"
- ⚠️ "Unusual transaction: ₹12,000 at Amazon on Apr 3 — is this correct?"
- 👤 "Based on your pattern, you'll spend ~₹18,500 next month"
- 💰 "You could save ₹1,800/month by reducing Swiggy orders by 30%"

STEP 6 — Financial Health Score

```
GET /api/insights/health → returns score 0-100
```

How the score is calculated:



● STEP 7 — Budget Alerts

User sets budget: Food = ₹5,000/month

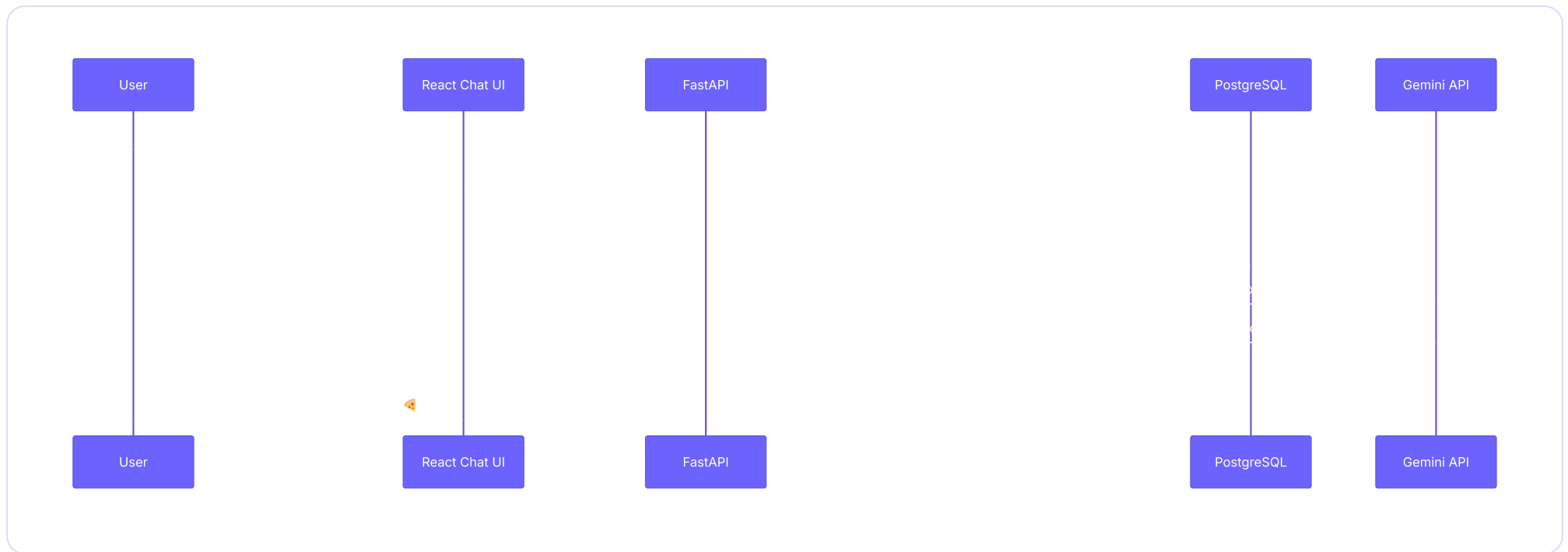
What happens when budget is nearing:

1. Every time a new expense is saved → Celery task checks budget
2. `SUM(expenses WHERE category='Food' AND month=current)` vs limit
3. If > 80% → send **push notification** via Firebase FCM
4. If > 100% → send **"LIMIT EXCEEDED"** alert
5. Alert stored in DB so it's not sent twice (`alert_sent = true`)

● STEP 8 — AI Chat Assistant

User types: "How much did I spend on food last month?"
or "Where can I save money?"
or "Show me my top 5 expenses this week"

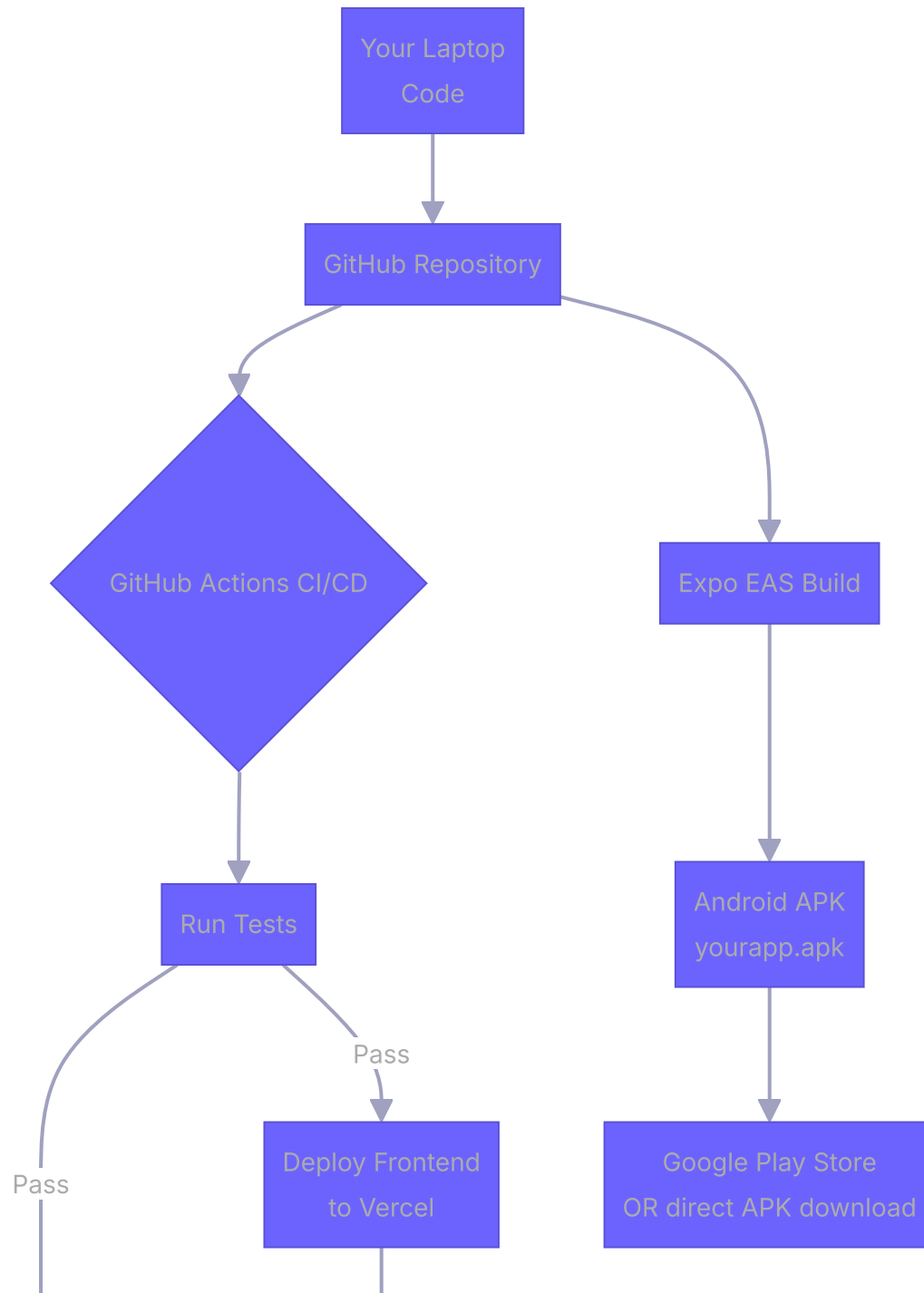
What happens (RAG — Retrieval Augmented Generation):

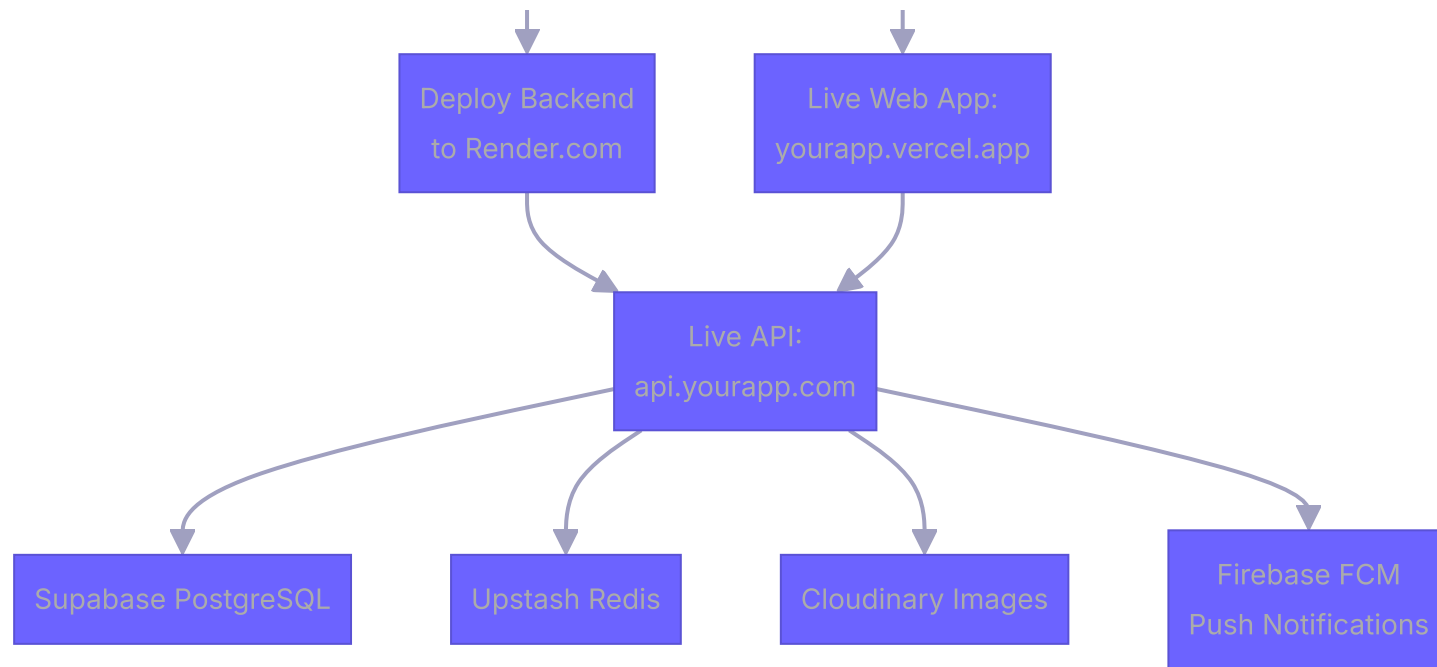


The AI doesn't guess — it **first queries your real data from DB**, then asks Gemini to phrase the answer naturally. This is called **RAG (Retrieval Augmented Generation)**.

PART 2 — DEPLOYMENT: Step by Step

FULL DEPLOYMENT PIPELINE





STEP-BY-STEP DEPLOYMENT GUIDE

DEPLOY STEP 1 — Set Up GitHub Repository

BASH

```
# On your laptop
git init
git add .
git commit -m "Initial commit: Smart Expense Tracker"
git remote add origin https://github.com/yourusername/smart-expense-tracker.git
git push -u origin main
```

- Create repo at **github.com**
- Keep it **public** (for portfolio visibility)
- Add a good README with screenshots

DEPLOY STEP 2 — Set Up Supabase (Free PostgreSQL)

1. Go to **supabase.com** → Create free account
2. Click "New Project" → name it `smart-expense-tracker`
3. Choose region: **Asia (Singapore)** — closest to India
4. Copy the **Connection String** from Settings → Database

```
postgresql://postgres:[password]@db.xxxx.supabase.co:5432/postgres
```

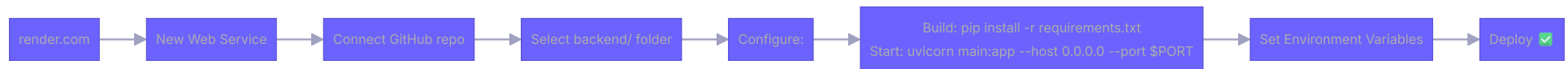
1. Run Alembic migrations to create tables:

BASH

```
cd backend  
alembic upgrade head # creates all tables in Supabase
```

1. Your database is now **live** ✅

DEPLOY STEP 3 — Deploy Backend to Render.com



Environment variables to set in Render:

ENV

```
DATABASE_URL=postgresql://postgres:xxx@db.supabase.co/postgres  
SECRET_KEY=your-super-secret-jwt-key-here  
GEMINI_API_KEY=AIZA ... your-key  
GOOGLE_CLIENT_ID=xxxxx.apps.googleusercontent.com  
GOOGLE_CLIENT_SECRET=xxxxx  
REDIS_URL=redis://default:xxx@upstash.io:6379  
CLOUDINARY_URL=cloudinary://xxx:xxx@yourcloud  
FIREBASE_KEY=path/to/firebase-key.json
```

After deploy:

- Render gives you a URL: <https://smart-expense-api.onrender.com>
 - Test it: <https://smart-expense-api.onrender.com/docs> → FastAPI Swagger UI
-

DEPLOY STEP 4 — Set Up Redis (Upstash — Free)

1. Go to **upstash.com** → Create account
2. "Create Database" → name it `expense-cache`
3. Select region: **AWS ap-south-1** (Mumbai)
4. Copy **REDIS_URL** → paste in Render environment variables

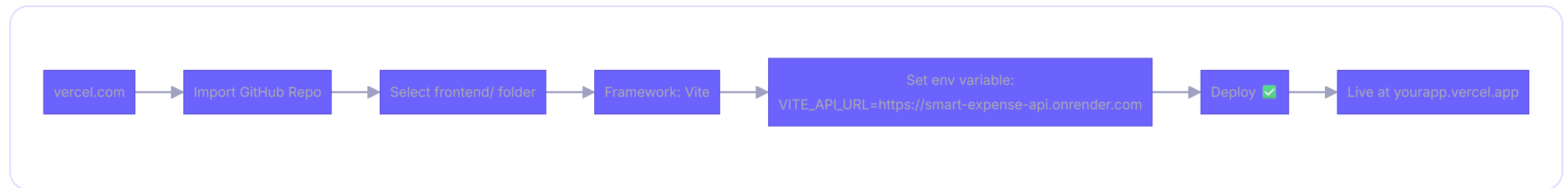
Used for: caching dashboard queries, rate limiting, Celery task queue

DEPLOY STEP 5 — Set Up Cloudinary (Image Storage)

1. Go to **cloudinary.com** → Free account
2. Copy your **Cloud Name, API Key, API Secret**
3. Set `CLOUDINARY_URL=cloudinary://api_key:api_secret@cloud_name` in Render

Used for: storing uploaded receipt images (25 GB free)

DEPLOY STEP 6 — Deploy Frontend to Vercel



Steps:

1. Go to **vercel.com** → Login with GitHub
2. "Import Project" → select your repo → set **Root Directory** to `frontend`
3. Add environment variable: `VITE_API_URL = https://smart-expense-api.onrender.com`
4. Click Deploy → Vercel builds and deploys automatically
5. Every `git push` to `main` → **auto-redeploys** ✓

Custom domain (optional):

- Vercel Settings → Domains → Add `expensetracker.yourdomain.com`
- Or use the free `.vercel.app` domain for portfolio

DEPLOY STEP 7 — Make it a PWA (Installable on Android)

Add these files to the React frontend:

`public/manifest.json` :

JSON

```
{
  "name": "Smart Expense Tracker",
  "short_name": "ExpenseAI",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#0f0f1a",
  "theme_color": "#6c63ff",
  "icons": [
    { "src": "/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icon-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

public/sw.js (Service Worker for offline):

JAVASCRIPT

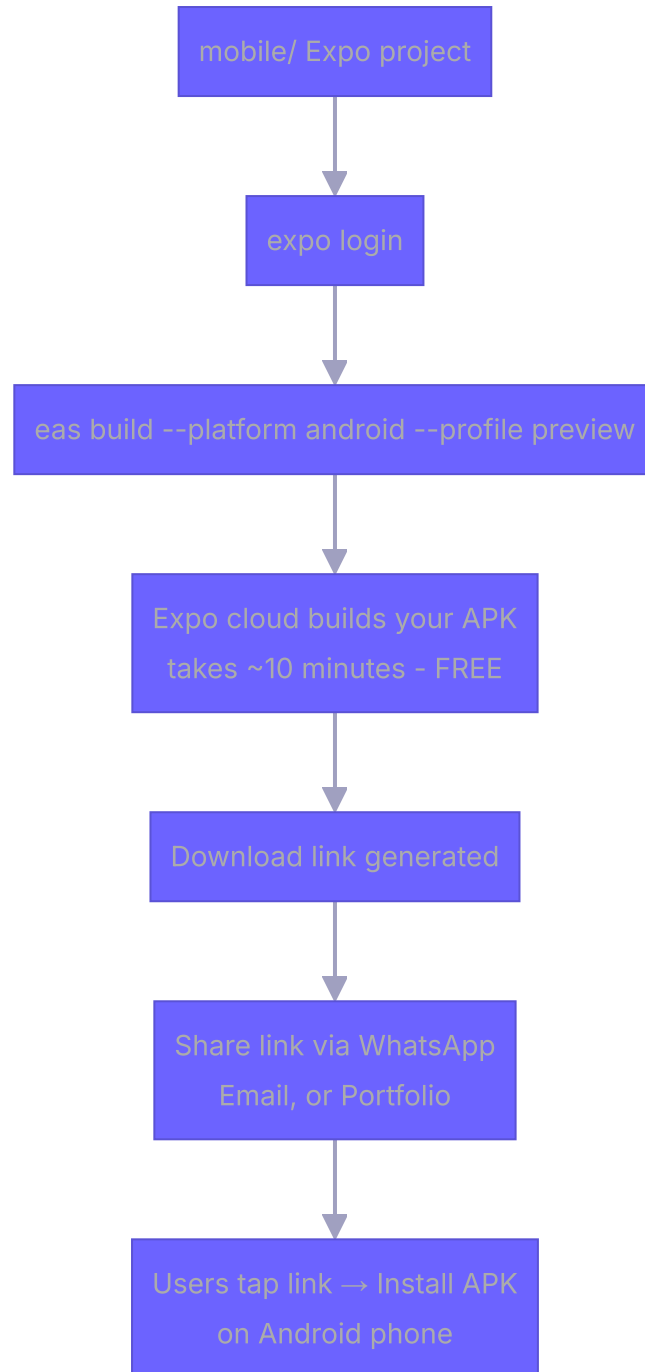
```
// Caches dashboard for offline viewing
self.addEventListener('install', (e) => {
  e.waitUntil(
    caches.open('expense-v1').then(cache =>
      cache.addAll(['/', '/index.html', '/assets/main.js'])
    )
  );
});
```

How user installs on Android:

1. Open **yourapp.vercel.app** in Chrome on Android

2. Chrome shows **"Add to Home Screen"** banner automatically
 3. Tap it → app icon appears on home screen
 4. Opens full-screen like a native app — **no Play Store needed!**
-

DEPLOY STEP 8 — Build Android APK (React Native / Expo)



Commands:

```
BASH

cd mobile
npm install -g eas-cli
eas login
eas build:configure

# Build APK for direct install (completely free)
eas build --platform android --profile preview
```

After build (~10 min):

- Expo gives you a **direct download link** for the `.apk` file
- Share it on WhatsApp, email, LinkedIn, or your portfolio/resume
- Anyone taps the link → clicks Install → app is on their phone ✅
- No Play Store. No \$25 fee. No approval wait.

DEPLOY STEP 9 — Firebase Push Notifications

1. Go to **console.firebase.google.com**
2. Create project → Add Android App → download `google-services.json`
3. Place in `mobile/android/app/`
4. In backend, download the **service account key JSON**
5. Set `FIREBASE_KEY` in Render environment

How notifications are sent:

PYTHON

```
# backend/services/notification.py
import firebase_admin
from firebase_admin import messaging

def send_budget_alert(user_fcm_token, category, percentage):
    message = messaging.Message(
        notification=messaging.Notification(
            title="🚨 Budget Alert",
            body=f"You've used {percentage}% of your {category} budget!"
        ),
        token=user_fcm_token
    )
    messaging.send(message)
```

DEPLOY STEP 10 — GitHub Actions CI/CD (Auto Deploy)

Create `.github/workflows/deploy.yml` :

YAML

```
name: Deploy Smart Expense Tracker

on:
  push:
    branches: [main]

jobs:
  test-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with: { python-version: '3.11' }
      - name: Install & Test
        run: |
          cd backend
          pip install -r requirements.txt
          pytest tests/ -v

  deploy-backend:
    needs: test-backend
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to Render
        run: curl -X POST ${ secrets.RENDER_DEPLOY_HOOK }

  deploy-frontend:
    needs: test-backend
```

```
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v3
  - name: Deploy to Vercel
    uses: amondnet/vercel-action@v25
    with:
      vercel-token: ${ secrets.VERCEL_TOKEN }
      vercel-org-id: ${ secrets.ORG_ID }
      vercel-project-id: ${ secrets.PROJECT_ID }
```

What this does:

- Every time you `git push` to `main` branch:
1.  Run all backend tests
 2. If pass → auto-deploy backend to Render
 3. If pass → auto-deploy frontend to Vercel
- Zero manual deployment needed after setup 
-



Final Deployment Checklist

Item	Service	Cost	Status
PostgreSQL Database	Supabase	Free	Configure
Redis Cache	Upstash	Free	Configure
Image Storage	Cloudinary	Free	Configure
Backend API	Render.com	Free	Deploy
Web App	Vercel	Free	Deploy
PWA (Android install)	Built into React	Free	Add manifest
Push Notifications	Firebase	Free	Configure
Android APK (direct share)	Expo EAS Build	Free	Build & share
Custom Domain	Vercel subdomain	Free	Optional
CI/CD Pipeline	GitHub Actions	Free	Configure
TOTAL COST		₹0	



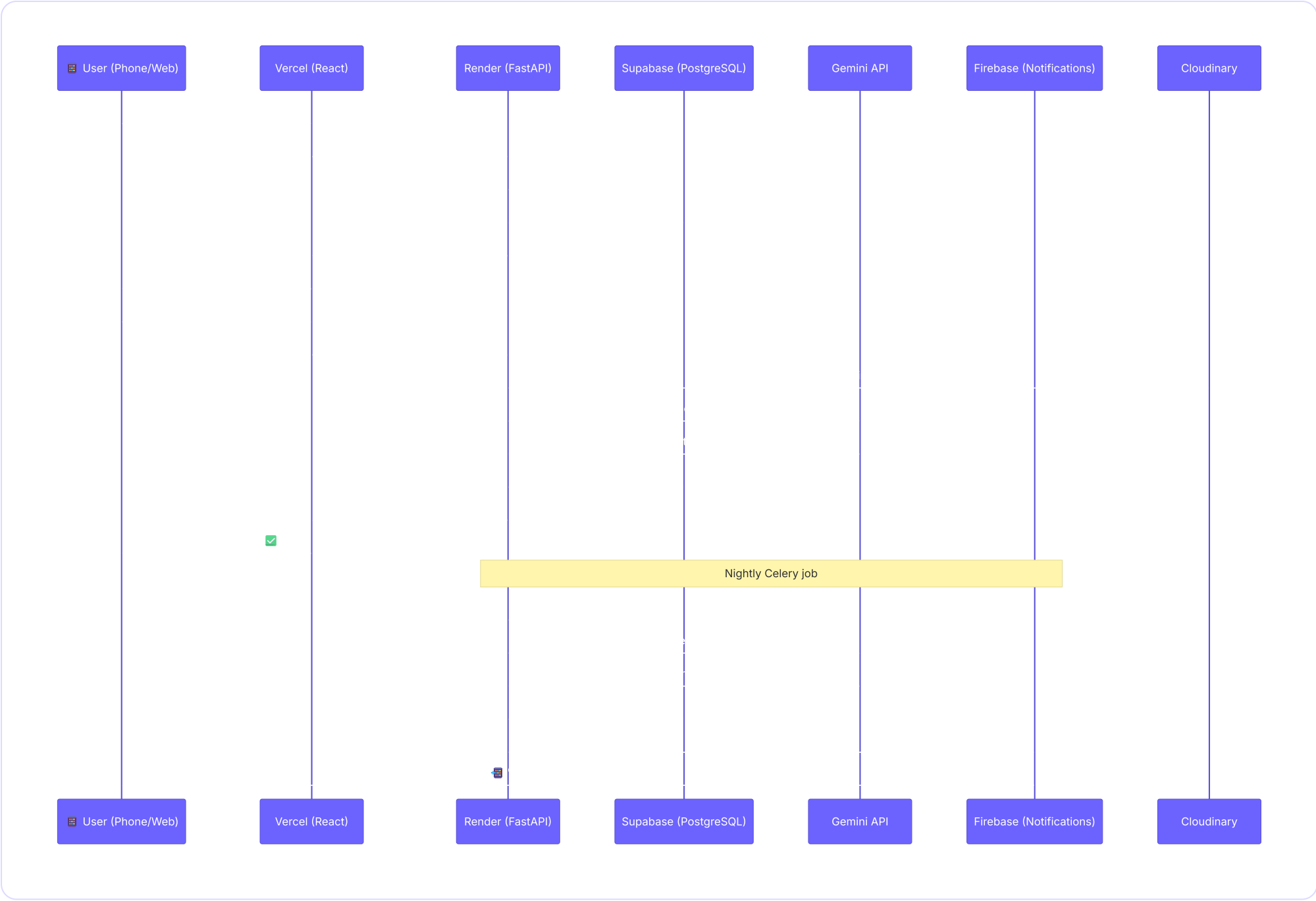
What Your URLs Look Like After Deployment

```
Web App (PWA):      https://smart-expense-tracker.vercel.app
Backend API:        https://smart-expense-api.onrender.com
API Docs (Swagger): https://smart-expense-api.onrender.com/docs
Android APK Link:   https://expo.dev/artifacts/eas/xxx.apk ← share this directly
```

💡 **PWA tip:** Users can also install directly from Chrome on Android without any APK — just open the web URL and tap "Add to Home Screen". This is your zero-friction mobile option.



Complete Flow Summary in One Diagram



🎯 **You now have a complete mental model** of every user interaction AND every deployment step. When you start building, refer back to this document for each phase.